

Parte 1 — Investigación y Diseño

1. Patrones de diseño para desacoplar la lógica de red (Separation of Concerns)

En React, un componente debería encargarse únicamente de:

Renderizar UI

Gestionar estado local

Responder a interacciones del usuario

La lógica de red (fetch, axios, peticiones HTTP) no forma parte de la responsabilidad del componente.

Por eso se extrae a servicios, siguiendo el principio Separation of Concerns.

¿Por qué es mala práctica hacer fetch directamente dentro de un useEffect? Aunque es muy común, tiene varios problemas:

1. Mezcla responsabilidades

El componente termina haciendo demasiadas cosas:

Renderiza

Gestiona estado

Hace peticiones HTTP

Maneja errores

Procesa datos

Esto lo hace difícil de mantener y de testear.

2. Dificulta el testing

Si el fetch está dentro del componente:

- No puedes mockearlo fácilmente
- Necesitas montar el componente entero para testear la lógica de red

En cambio, si la lógica está en un servicio:

```
import { getMovies } from "../services/api";
```

Puedes testear getMovies() de forma aislada.

3. Duplica lógica

Si varios componentes necesitan la misma petición, acabarás copiando el mismo fetch en varios sitios.

Con un servicio, la lógica está centralizada.

4. Hace más difícil manejar errores y reintentos

Un servicio puede tener:

1. Retries
2. Timeouts
3. Interceptores
4. Transformación de datos

Un componente no debería encargarse de eso.

5. Evita problemas de renderizados múltiples

React 18 ejecuta algunos efectos dos veces en modo estricto.

Si el fetch está dentro del useEffect, puedes provocar:

1. Peticiones duplicadas
2. Estados inconsistentes

Un servicio desacoplado reduce estos problemas.

3. Concepto de CORS (Cross-Origin Resource Sharing)

CORS es un mecanismo de seguridad del navegador que bloquea peticiones HTTP entre orígenes distintos, por ejemplo:

- Frontend: `http://localhost:5173`
- Backend: `http://localhost:3000`

Como los dominios son distintos, el navegador bloquea la petición a menos que el backend autorice explícitamente al frontend.

¿Qué ocurre si el backend NO autoriza tu origen?

Tu servicio fallará con un error como:

Código

```
Access to fetch at 'http://localhost:3000/api/movies'
from origin 'http://localhost:5173' has been blocked by CORS policy
```

Esto significa que el backend no ha enviado el header:

Código:

```
Access-Control-Allow-Origin: http://localhost:5173
```

Sin ese header, el navegador bloquea la respuesta antes de que llegue a tu código.